TaskForge — Day 2 Notes

Day 2: CRUD APIs and In-Memory Storage

Project: TaskForge · Backend: Python + FastAPI

Main objective: Understand how an API receives a request, processes data, stores a job, and returns a response.

Topics covered:

- Unique IDs using UUIDs
- In-memory dictionary storage
- Create, read, update, and list jobs
- HTTP status codes and 404 errors
- Query parameters and pagination
- Request validation and API testing

1. What did we build on Day 2?

Imagine TaskForge is a service where users submit jobs for processing. For example, a user submits a report-generation job and wants to check its status later.

The API needs to support these operations:

Operation	Purpose	HTTP method
Create a job	Add a new job	POST
Get a job	Retrieve one job by ID	GET
List jobs	Retrieve multiple jobs	GET
Update a job	Change its status or details	PATCH

These are the foundations of CRUD:

- C — Create: Add new data.
- R — Read: Retrieve existing data.
- U — Update: Modify existing data.
- D — Delete: Remove data.

We focused on creating, reading, updating, and listing jobs.

2. Generating unique IDs with UUID

Every job needs an identifier so that the API can distinguish it from other jobs.

Python provides the built-in `uuid` module.

```
import uuid

job_id = str(uuid.uuid4())

print(job_id)
```

Example output:

```
7a31e8b2-6e29-4a01-9d75-1b3f2d8c6a40
```

The output will differ each time you run the code.

Understanding the code

- `import uuid` imports Python's UUID utilities.
- `uuid.uuid4()` generates a random UUID.
- `str(...)` converts the UUID object into a string.

Why use UUIDs? If you create multiple jobs, each needs its own ID. UUIDs make collisions extremely unlikely without requiring you to manually assign IDs.

3. Understanding in-memory storage

Before connecting TaskForge to a database, we used a Python dictionary to store jobs temporarily.

```
jobs = {}
```

This creates an empty dictionary.

A job can be stored like this:

```
jobs["job-101"] = {
    "id": "job-101",
    "task": "Generate a report",
    "status": "pending"
}
```

Now the dictionary contains one job.

To retrieve it:

```
print(jobs["job-101"])
```

To check whether an ID exists safely:

```
job = jobs.get("job-101")

if job is not None:
    print(job)
else:
    print("Job not found")
```

Why did we use a dictionary?

The job ID acts as the key, and the job details act as the value. This lets the API find a job directly by its ID.

Important limitation: In-memory storage is temporary. Restarting the server normally clears the dictionary. That is why Day 3 moves us toward PostgreSQL, which stores data persistently.

4. Creating a job — POST endpoint

A `POST` endpoint allows a client to send information to the server to create a new resource.

A simplified FastAPI example:

```
from fastapi import FastAPI
from pydantic import BaseModel
import uuid

app = FastAPI()

jobs = {}

class JobCreate(BaseModel):
    task: str

@app.post("/jobs", status_code=201)
def create_job(job_data: JobCreate):
    job_id = str(uuid.uuid4())

    job = {
        "id": job_id,
        "task": job_data.task,
        "status": "pending"
    }

    jobs[job_id] = job

    return job
```

Explanation

- `FastAPI()` creates the application.
- `jobs = {}` creates temporary storage.
- `JobCreate` defines the expected request body.
- `@app.post("/jobs")` registers the endpoint.
- `job_data: JobCreate` tells FastAPI to validate the incoming data.
- `uuid.uuid4()` generates a unique ID.
- `jobs[job_id] = job` stores the job.
- `return job` sends the created job back to the client.
- `status_code=201` indicates that a resource was created.

Example request body:

```
{
  "task": "Generate a report"
}
```

Example response:

```
{
  "id": "generated-unique-id",
  "task": "Generate a report",
  "status": "pending"
}
```

The ID shown here is illustrative; the actual API generates a UUID.

Why use Pydantic?

Without request validation, the application would need to manually check whether the incoming JSON has the required fields and correct data types.

Pydantic helps FastAPI validate incoming data before the endpoint processes it.

5. Retrieving a job — GET endpoint

Once a job has been created, the client needs to retrieve it.

```
from fastapi import HTTPException

@app.get("/jobs/{job_id}")
def get_job(job_id: str):
    job = jobs.get(job_id)

    if job is None:
        raise HTTPException(
            status_code=404,
            detail="Job not found"
        )

    return job
```

Understanding the code

- `"/jobs/{job_id}"` defines a path parameter.
- `job_id: str` receives the requested identifier.
- `jobs.get(job_id)` looks for that job in the dictionary.
- If no job exists, the API raises an HTTP exception.
- Otherwise, it returns the job.

For example:

```
GET /jobs/7a31e8b2-6e29-4a01-9d75-1b3f2d8c6a40
```

A successful request returns the job. A request for a nonexistent ID returns:

```
{
  "detail": "Job not found"
}
```

with HTTP status `404`.

Why is 404 important?

An API should clearly communicate when a requested resource does not exist. Returning an error is better than pretending the operation succeeded.

6. Updating a job — PATCH endpoint

A job's status may change as processing progresses.

For example:

`pending → processing → completed`

A `PATCH` request changes selected fields instead of replacing the entire resource.

```
from typing import Literal
from pydantic import BaseModel

class JobUpdate(BaseModel):
    status: Literal["pending", "processing", "completed", "failed"]

@app.patch("/jobs/{job_id}")
def update_job(job_id: str, update_data: JobUpdate):
    job = jobs.get(job_id)

    if job is None:
        raise HTTPException(
            status_code=404,
            detail="Job not found"
        )

    job["status"] = update_data.status

    return job
```

Example request body:

```
{
  "status": "completed"
}
```

What does `Literal` do?

```
Literal["pending", "processing", "completed", "failed"]
```

It restricts the accepted value to one of the specified options.

For example, `"completed"` is valid, but `"finished"` is rejected because it is not in the allowed set.

Why this matters: Backend validation prevents invalid states from entering your application.

7. Listing all jobs

A user may want to see all the jobs currently stored in TaskForge.

```
@app.get("/jobs")
def list_jobs():
    return list(jobs.values())
```

Understanding the code

- `jobs.values()` gives you the stored job objects.
- `list(...)` converts those values into a list.
- FastAPI serializes the list into a JSON response.

If the dictionary contains three jobs, the response will contain three job objects.

This is useful for testing, but a production API should usually support pagination rather than returning an unlimited number of records.

8. Pagination — returning data in manageable portions

Imagine TaskForge has 10,000 jobs. Returning all of them in one response would waste bandwidth and resources.

Pagination allows the client to request a smaller portion of the data.

Two common query parameters are:

- `limit`: maximum number of jobs to return.
- `offset`: number of jobs to skip before returning results.

Example:

```
GET /jobs?limit=10&offset=20
```

This requests up to 10 jobs after skipping the first 20.

A simplified implementation:

```
@app.get("/jobs")
def list_jobs(limit: int = 10, offset: int = 0):
    all_jobs = list(jobs.values())

    return all_jobs[offset:offset + limit]
```

Understanding Python slicing

```
all_jobs[offset:offset + limit]
```

Suppose:

```
offset = 20
limit = 10
```

Python evaluates the slice as:

```
all_jobs[20:30]
```

It returns up to 10 items, starting at index 20.

Validating pagination parameters

We don't want clients requesting negative limits or negative offsets.

FastAPI provides `Query` for validation:

```
from fastapi import Query

@app.get("/jobs")
def list_jobs(
    limit: int = Query(default=10, ge=1, le=100),
    offset: int = Query(default=0, ge=0)
):
    all_jobs = list(jobs.values())

    return all_jobs[offset:offset + limit]
```

Here's what the constraints mean:

Parameter	Constraint	Meaning
limit	ge=1	At least 1
limit	le=100	At most 100
offset	ge=0	Cannot be negative

`ge` means greater than or equal to, while `le` means less than or equal to.

Invalid values are rejected by FastAPI with a validation error instead of being accepted silently.

9. Understanding HTTP status codes

HTTP status codes tell the client what happened when it made a request.

Code	Meaning	Example in TaskForge
200 OK	Request succeeded	Job retrieved or updated
201 Created	Resource created	New job created
404 Not Found	Resource doesn't exist	Invalid job ID
422 Unprocessable Entity	Request validation failed	Invalid status or pagination input
500 Internal Server Error	Unexpected server failure	Unhandled application error

Don't memorize every HTTP status code right now. Understand these common ones and when to use them.

10. Testing the API

FastAPI automatically provides interactive API documentation through Swagger UI.

Start your server from your project directory, with your virtual environment activated:

```
uvicorn main:app --reload
```

This assumes your FastAPI instance is named `app` in `main.py`. If your project uses a different filename or package structure, use the corresponding module path.

Open this address in your browser:

`http://127.0.0.1:8000/docs`

Test these operations in order:

Test 1 — Create a job

`/jobs`

Send a valid task name. Confirm that the response contains an ID and a `pending` status.

Test 2 — Retrieve the job

`/jobs/{job_id}`

Copy the ID from Test 1 and confirm the same job is returned.

Test 3 — Update the status

`/jobs/{job_id}`

Change the status to `processing` and verify the response.

Test 4 — List and paginate

`/jobs?limit=10&offset=0`

Check that the API returns a list and that pagination values are validated.

Test 5 — Check error handling

Try a nonexistent job ID, an invalid status, and a negative pagination value. Confirm the appropriate errors are returned.

11. Understand the complete request-response flow

This is one of the most important concepts from Day 2.

1. Client sends a request

Example: POST /jobs

2. FastAPI validates the input

Checks the request body and parameters

3. Endpoint runs Python logic

Generates an ID and creates or updates a job

4. Job is stored or retrieved

Day 2: in-memory dictionary

5. API returns a response

JSON data or an appropriate HTTP error

Remember this flow. When debugging a backend, ask yourself which step is failing.

12. Common mistakes and how to debug them

Problem	Likely reason	What to check
ModuleNotFoundError	Missing package or wrong environment	Activate .venv and check installed packages
404 Not Found	Wrong URL or missing job	Check the route and job ID
422 Unprocessable Entity	Invalid request body or query parameters	Check field names, types and constraints
405 Method Not Allowed	Wrong HTTP method	Use the method registered for that route
Job disappears after restart	In-memory storage was reset	Persistent database is needed
Changes don't appear	Wrong file or server process	Check the project structure and reload logs

One important distinction: `404` may mean that a route doesn't exist or that the requested job doesn't exist. Read the response to distinguish between them.

13. What you should understand without looking at the notes

Test your understanding by explaining these in your own words:

- Why does every job need a unique ID?
- Why did we use a dictionary before PostgreSQL?
- What is the difference between `POST` and `GET`?
- Why do we use `PATCH` to update a job?
- What does `HTTPException(status_code=404)` do?
- Why does FastAPI use Pydantic models?
- What do `limit` and `offset` mean?
- Why should an API validate its query parameters?
- Why is in-memory storage unsuitable for permanent job data?
- How would you test a job-creation endpoint?

If you can explain these concepts and write small versions of the endpoints yourself, you are making progress toward independently building backend applications—not just copying code.

14. Save Day 2 progress to GitHub

Once your implementation and tests are working, open Terminal in your TaskForge project directory and run:

```
git status
git add .
git commit -m "feat: implement job CRUD and pagination"
git push
```

Review `git status` before staging, and make sure `.env`, `.venv`, and other ignored files are not being committed. If your remote branch is configured to track `main`, `git push` should upload your commit.

15. What comes next — Day 3

Day 3: PostgreSQL and Persistent Storage

On Day 2, jobs live in memory. On Day 3, the goal is to store jobs in a real database so they survive server restarts.

Planned concepts:

- Relational databases and tables
- PostgreSQL setup and connection
- Database models and schemas
- CRUD operations backed by PostgreSQL
- Environment variables for database configuration
- Testing persistence across server restarts

My advice: Don't rush into Day 3 just to finish the roadmap. First, make sure you can explain Day 2 and reproduce the core logic without copying every line. You can still use documentation and look up syntax—that's normal engineering practice.

One final distinction: these notes describe the Day 2 concepts and representative FastAPI implementations. Your actual project may organize routes, schemas, and storage differently. Follow your working TaskForge code where it differs.